

SpendIQ: AI & Data Engineer Challenge

Company: techtorch **Role:** AI & Data Engineer (Mid / Senior) **Format:** Take-home, 2 hours, self-paced **Interview format:** Remote, live over Microsoft Teams with screen share **Deliverable:** A runnable end-to-end agent with a simple UI, plus short design notes **Submission:** Share a zipped repo (or a private Git link) with us at least one hour before your scheduled Teams interview

1. Scenario

You have been engaged by **Nimbus Retail Co.**, a midsize retailer (~1,500 employees, ~USD 220M annual revenue). Finance and Procurement want a conversational copilot that can answer natural-language questions over their vendor, contract, purchase order, and invoice data, and can cross-reference the contents of contract and policy documents.

Today, a Finance analyst has to pivot between the ERP, a contracts folder, and Excel to answer questions like “Which vendors have auto-renewing contracts expiring in the next 90 days?” or “Are we paying any invoices on terms shorter than the contract allows?” The client wants that to feel like asking a coworker.

Your job is to build a **proof of concept** of that copilot in two hours.

2. Objective

Build an agentic system that:

1. **Ingests** the provided structured data (CSVs) into a database of your choice and the provided unstructured documents (contract files and policies) into a retrieval store of your choice.
2. **Answers** natural-language questions that require reasoning across structured data, unstructured documents, or both.
3. **Explains** itself by surfacing which tool(s) and which source(s) were used, ideally with citations to specific invoice IDs, contract clauses, or documents.
4. **Runs** end-to-end from a simple UI (chat is fine). During the Teams interview you will share your screen and the interviewer will ask you to type questions directly into your demo.

This is a proof of concept, not a production system. We care more about the *quality of your thinking* than the *completeness of your feature list*.

3. What's in the Package

SpendIQ/

CANDIDATE_BRIEF.md	(this document)
CANDIDATE_BRIEF.pdf	(PDF rendering of this document)
DATA_DICTIONARY.md	(schema reference + ER sketch)
data/	(structured data in CSV)
departments.csv	
vendors.csv	
contracts.csv	
purchase_orders.csv	
invoices.csv	
docs/	
contracts/	(15 contract documents: 12 Markdown, 3 PDF)

policies/
README.md

(3 policy documents in Markdown)
(package overview)

Reference date: Treat *today* as **2026-04-21** (the data is anchored to this date so queries like “*expiring in the next 90 days*” return meaningful results).

4. Ground Rules

Recommended stack (not enforced)

Use whatever you are fastest with, but the team is optimized around:

- **Backend:** Python
- **Database:** Postgres (with pgvector if you want hybrid retrieval)
- **Frontend:** Next.js / React (Vite is fine too)
- **LLM framework:** Your choice (LangGraph, LlamaIndex, Pydantic AI, raw SDK, etc.)
- **LLM provider:** Your choice. Bring your own API key (OpenAI, Anthropic, Bedrock, or a local model).

If you use a different stack, we’ll still evaluate fairly. What matters is that it runs end-to-end on your own machine during the Teams screen-share demo.

AI coding assistants are welcome

You are encouraged to use Claude Code, Codex, Cursor, or similar. Part of what we are hiring for is the ability to move fast with AI assistants. Please be prepared to talk through:

- Where the assistant helped and where it led you astray
- Prompts that worked well for you
- What you verified yourself vs. accepted at face value

Two hours, one attempt

We recommend setting a strict timer. Use the full two hours. Submit whatever you have at the end, even if it is rough. A partially-working system with clear thinking beats a polished demo with weak design choices.

Be honest about what’s not working

If something is broken or mocked, say so in your notes. We score honesty higher than over-claiming.

5. Sample Questions the Agent Should Handle

These are **illustrative**. The interviewer will ask similar questions (and some unseen ones) during the walk-through. You don’t need to pre-can them; a well-designed agent will handle most without hardcoding.

Tier 1: Structured data only

- What is our total spend with *Stratos Cloud Services* over the last 12 months?
- Who are our top 10 vendors by year-to-date spend?
- How many active contracts do we have, broken down by category?

Tier 2: Document-only

- Summarize the auto-renewal terms for our Orion Logistics contract.
- What notice period is required to exit the Aperture SaaS subscription?
- What does our procurement policy say about splitting purchase orders?

Tier 3: Cross-source reasoning

- Which contracts auto-renew in the next 90 days and what is the notice window for each? (*Structured dates plus contract clauses.*)
- Find invoices whose billed payment terms are shorter than the terms stated in the underlying contract. (*Invoices plus contract docs.*)
- Identify purchase orders that appear to have been split to stay under approval thresholds. (*POs plus procurement policy.*)
- Which vendors have more than USD 50,000 in year-to-date spend but no active contract? (*Invoices/POs plus contracts table.*)

Tier 4: Stretch (good signal for Senior)

- Flag contracts where cumulative invoice spend has exceeded the stated total contract value cap.
- Draft a one-paragraph renegotiation brief for the three vendors where we have the most leverage (highest spend, contract expiring, no auto-renew protection).
- Surface duplicate-looking invoices from the same vendor in the last six months.

6. What to Build (minimum bar)

To be interview-ready, you need the following working end-to-end:

1. **Ingestion pipeline.** A script or flow that loads the CSVs into your database and the documents into your retrieval store. Should be reproducible (make ingest, python -m spendiq.ingest, or similar).
2. **Agent with at least two tools.** Typically a SQL/structured-data tool and a document-retrieval tool. Anything beyond that (arithmetic, date utilities, specialized search) is bonus.
3. **Chat UI.** Simple is fine. A text input, a running conversation, and something that shows which source was used is plenty.
4. **A short README / design notes** (can be in DESIGN_NOTES.md) covering: architecture, data-modeling decisions, chunking/embedding choices, how tools were exposed to the agent, limitations, and what you would do with more time.

7. Suggested 2-Hour Milestone Plan

The exact pacing is yours; this is a reference pacing we have seen work well:

Time	Milestone
0:00 to 0:15	Read this brief, skim the data dictionary and a few docs, sketch your architecture
0:15 to 0:45	Stand up the DB, ingestion script, and load CSVs; start the document embedding job
0:45 to 1:15	Build the agent skeleton, wire up the SQL and document-retrieval tools
1:15 to 1:45	Wire the UI, verify end-to-end on 4 or 5 sample questions from each tier

Time	Milestone
1:45 to 2:00	Write design notes, clean up, submit

8. Submission Checklist

- ☐ Repo builds/runs from a clean clone with the commands documented in your README
- ☐ Ingestion reruns from scratch (we will reset and re-ingest to verify)
- ☐ Agent answers at least a handful of questions end-to-end via the UI
- ☐ Design notes cover architecture, decisions, limitations, and next steps
- ☐ `.env.example` lists any required environment variables (we can provide a short-lived Anthropic or OpenAI key at the interview if you need one)

9. Interview Walk-Through (remote, via Microsoft Teams)

The interview runs on Microsoft Teams. You will receive a calendar invite with the Teams link. Plan to spend ~60 minutes total:

- **5 min.** Architecture overview
- **15 min.** Live demo over screen share. The interviewer will also ask a few unseen questions and occasionally push into territory your agent was not designed for.
- **20 min.** Deep dive on key decisions (data modeling, chunking, tool design, retrieval, prompts, error handling)
- **10 min.** Retrospective: what worked, what broke, role of the AI assistant, what you'd do differently with more time
- **10 min.** Your questions for us

Before the call

- Join from a machine where your full stack runs end-to-end (DB, backend, UI, AI assistant).
- Have the app pre-started and sanity-checked 5 minutes before the call so we don't burn demo time on boot issues.
- Have your code editor and, if you used one, your AI coding assistant open. We may ask you to show a prompt or a diff live.
- Have a stable internet connection and a working mic. Camera is preferred but not required.
- If you need a short-lived Anthropic or OpenAI key because your own key has rate-limited, let us know in your submission email and we will share one via Teams chat at the start of the call.

10. How We Evaluate

We evaluate on five dimensions (see our rubric for the breakdown across Mid vs Senior):

1. **Agentic design.** Tool decomposition, routing, handling of ambiguous questions.
2. **Data engineering.** Schema modeling, ingestion robustness, chunking and retrieval choices, hybrid retrieval where it helps.
3. **Code and architecture quality.** Separation of concerns, reproducibility, reasonable error handling.
4. **Deliverable and demo.** Does it run? Is the UX usable? Are sources cited?
5. **Production thinking.** What would break at 10x scale, cost/latency awareness, eval and guardrail instincts (weighted higher for Senior).

We explicitly do **not** evaluate on:

- Visual polish beyond “it’s usable”
- Number of features beyond the minimum bar
- Whether you got every planted edge case. We care that you had a principled approach.

Good luck. Have fun. Don’t overbuild.